

PYTHON CONSOLE PROJECT

나만의 프롬프트 관리 프로그램

개발 과정 및 최종 결과 보고서

구분	내용
프로젝트명	prompt-manager
개발 언어	Python 3.14.6
개발 도구	Visual Studio Code, Git, GitHub
GitHub 저장소	https://github.com/coolgirl2046/prompt-manager
작성일	2026 년 8 월 3 일

필수 기능 + 보너스 1(JSON 저장·불러오기 및 Markdown 내보내기) + 평가 보완 완료

1. 프로젝트 개요

이 프로젝트에서는 터미널에서 메뉴 번호를 입력하여 프롬프트를 관리하는 Python 프로그램을 제작하였습니다. 프롬프트 추가, 전체 목록 보기, 카테고리별 보기, 검색, 상세 보기, 즐겨찾기 추가·해제와 즐겨찾기 목록 보기 기능을 사용할 수 있습니다.

필수 기능은 따로 설치해야 하는 프로그램 없이 Python 기본 문법만 사용하여 구현하였습니다. 프롬프트는 리스트와 딕셔너리에 저장하였으며, 각 프롬프트에는 제목, 내용, 카테고리, 즐겨찾기 여부가 들어 있습니다. 보너스 기능으로 JSON 저장·불러오기와 카테고리별 Markdown 내보내기를 추가하였습니다. 따라서 프로그램을 종료한 후 다시 실행해도 추가한 내용과 즐겨찾기 상태가 유지됩니다.

필수 기능에서는 프로그램 실행 중에 데이터를 유지하도록 요구합니다. 이 프로젝트는 보너스 1 을 추가로 구현하여 프로그램을 종료한 후에도 프롬프트와 즐겨찾기 상태가 JSON 파일에 유지되도록 확장하였습니다.

최종 결과 프롬프트 관리 프로그램의 필수 기능을 모두 구현하였습니다. GitHub 저장소 업로드, 16 개 이상의 기능·문서 단위 커밋과 기능 브랜치 병합을 완료하였습니다. 보너스 기능으로 JSON 저장·불러오기와 Markdown 내보내기를 구현하였으며, 중복 제목 방지 기능도 추가하였습니다.

2. 과제 요구사항 충족 여부

평가 영역	상태	확인 결과
Python 콘솔 프로그램	충족	번호 선택형 메뉴와 종료·재표시 흐름 구현
기본 프롬프트	충족	이전 미션 자료를 활용한 기본 데이터 3 개 등록
필수 기능	충족	추가·목록·카테고리·검색·상세·즐거찾기 구현
데이터 구조	충족	리스트와 딕셔너리 사용

평가 영역	상태	확인 결과
코드 구조	충족	기능별 함수 분리
Git 커밋	충족	스크린샷에서 16 개 확인, 이후 README 보완 커밋 추가
브랜치·병합	충족	feature 브랜치 생성 및 main 병합
GitHub	충족	공개 저장소 main 브랜치 업로드
README	충족	설명·실행 방법·기능·카테고리 작성
보너스 1	완료	종료 후에도 남는 JSON 저장·불러오기와 Markdown 내보내기

3. 개발 환경 설정

VSCode 에서 프로젝트 폴더를 열고 Python 파일을 작성하고 실행하였습니다. 터미널에서 Python 과 Git 버전을 확인하고 Git 사용자 이름과 이메일을 설정하였습니다. Python 3.14.6 을 사용하여 과제 기준인 Python 3.10 이상을 충족하였습니다.

VSCode 에 Python 확장을 설치하여 Python 파일을 실행하였습니다. Git 기본 브랜치는 main 으로 설정하였으며, GitHub 저장소와 연결하여 최종 코드를 업로드하였습니다.

항목	확인 내용
코드 편집기	Visual Studio Code
Python 버전	Python 3.14.6
Python 실행 환경	컴퓨터에 설치된 기본 환경 사용, venv/pyenv 사용 안 함
외부 라이브러리	사용하지 않음
Git 버전	git version 2.55.0.windows.3
Git 사용자 이름	coolgirl2046
Git 사용자 이메일	coolgirl2046@naver.com
기본 브랜치	main
실행 명령	python prompt_manager.py
프로젝트 위치	C:\Users\Wpc\Desktop\prompt-manager

확인에 사용한 명령은 `python --version`, `git --version`, `git config --global user.name`, `git config --global user.email` 입니다. 명령 결과를 표와 그림 1 의 주석에도 글자로 적어 그림을 읽지 못하더라도 설정 값을 확인할 수 있게 하였습니다.

venv 와 pyenv 는 프로젝트마다 Python 실행 환경을 나누어 관리할 때 사용하는 도구입니다. 이 프로그램은 외부 라이브러리가 없는 기초 프로젝트이므로 별도의 환경 관리 도구를 사용하지 않았습니다.

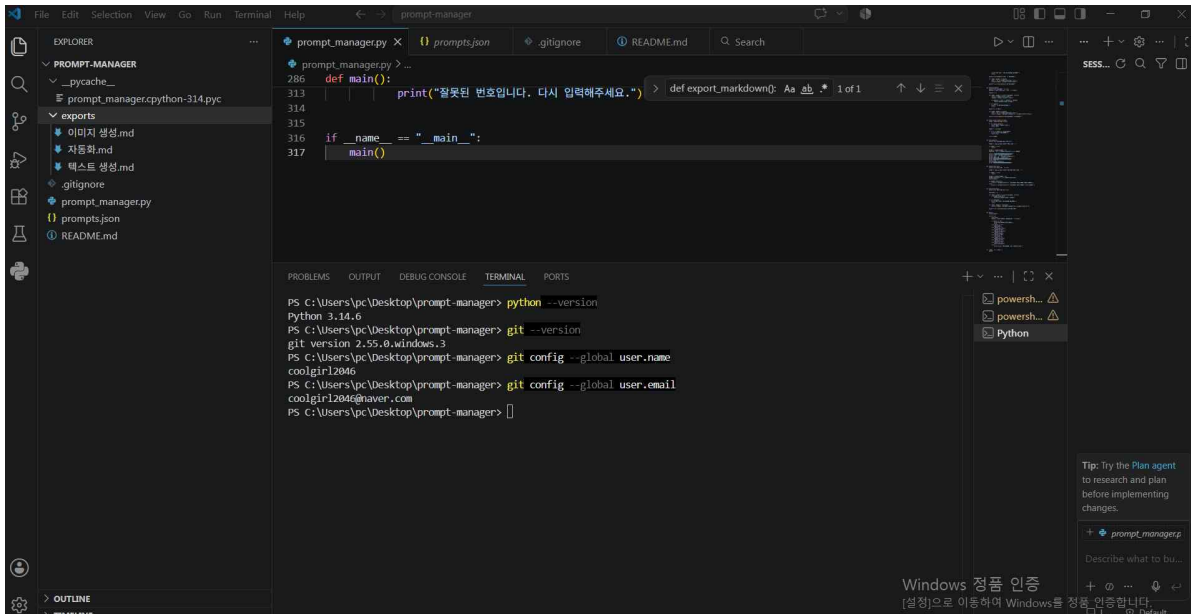


그림 1. VSCode 프로젝트 화면과 Python-Git 개발 환경 설정

주석 VSCode 에서 prompt-manager 폴더를 연 화면입니다. python --version 으로 Python 3.14.6, git --version 으로 Git 2.55.0.windows.3 을 확인하였습니다. Git 사용자 이름은 coolgirl2046, 이메일은 coolgirl2046@naver.com 입니다.

3.1 Python 기본 실행 확인

hello.py 파일에 print("Hello") 코드를 작성하고 터미널에서 실행하였습니다. 이를 통해 Python 파일 생성과 실행 환경이 정상적으로 작동하는 것을 확인하였습니다.

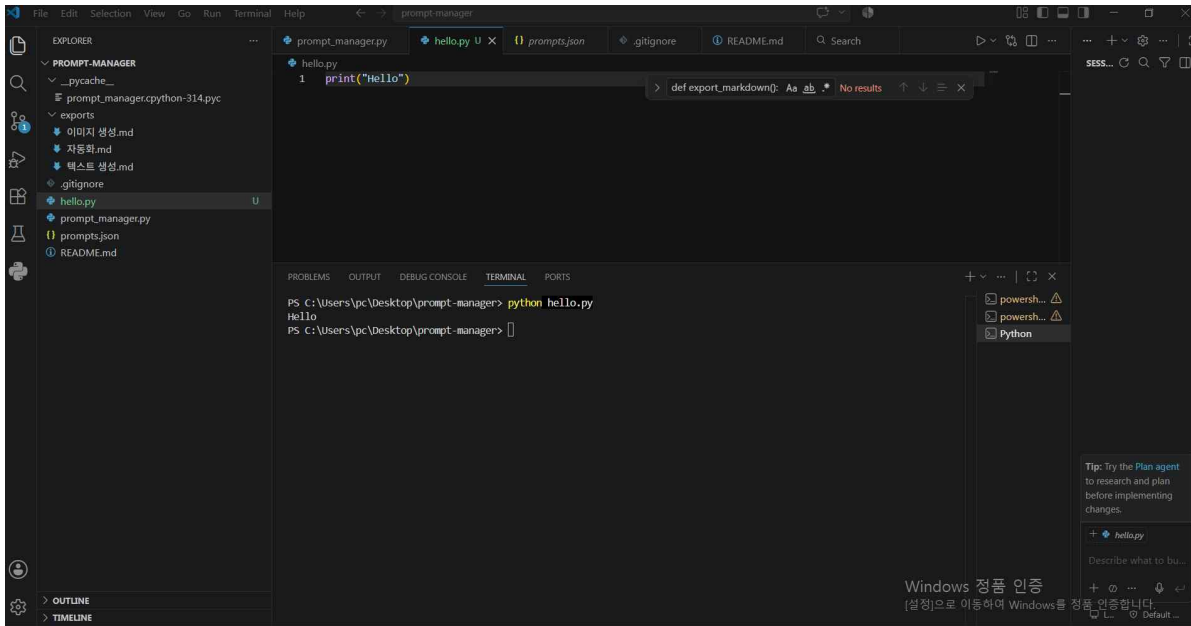


그림 2. hello.py 코드와 Hello 실행 결과

주석 hello.py 파일에 `print("Hello")`를 작성하고 실행한 화면입니다. 터미널에 Hello가 출력되어 Python 파일이 정상적으로 실행되는 것을 확인하였습니다.

4. 프로그램 구성

4.1 프롬프트 데이터 구성

프로그램은 prompts라는 리스트 안에 여러 개의 딕셔너리를 넣는 방식으로 구성하였습니다. 딕셔너리 하나가 프롬프트 하나를 뜻합니다. title은 제목, content는 내용, category는 카테고리, favorite는 즐겨찾기 상태를 저장합니다. 기본 카테고리는 텍스트 생성, 이미지 생성, 영상 생성, 페르소나, 자동화, 기타입니다.

이름	저장하는 내용	값의 종류
title	프롬프트 제목	문자열
content	프롬프트 전체 내용	문자열
category	프롬프트 분류	문자열
favorite	즐거찾기함 또는 하지 않음	참-거짓(True/False)

비교 항목	리스트	딕셔너리
역할	여러 프롬프트를 순서대로 저장	프롬프트 한 개의 정보를 이름별로 저장
장점	번호를 붙이고 순서대로 출력하기 쉬움	제목·내용·카테고리를 구분하기 쉬움
단점	데이터가 많아지면 검색이 느려질 수 있음	키 이름을 정확하게 사용해야 함
사용 예	<code>prompts[0]</code>	<code>prompt["title"]</code>
선택 이유	전체 프롬프트 목록 관리에 적합	프롬프트 한 개의 정보 관리에 적합

현재처럼 프롬프트가 5 개 정도인 작은 목록은 처음부터 차례로 확인해도 부담이 작습니다. 하지만 데이터가 수천 개로 늘어나면 매번 전체 리스트를 비교해야 하므로 검색이 느려질 수 있습니다. 이 프로젝트는 초보자가 이해하기 쉬운 구조와 작은 데이터 규모를 고려하여 리스트와 딕셔너리를 선택하였습니다.

카테고리를 추가하거나 삭제할 때는 `prompt_manager.py` 의 `CATEGORIES` 목록을 수정하고, 기존 `prompts.json` 에 저장된 `category` 값도 함께 확인합니다.

카테고리 이름을 바꿔도 기존 JSON 데이터는 자동으로 변환되지 않습니다. 따라서 기존 프롬프트의 `category` 값은 `prompts.json` 에서 새 이름으로 직접 확인·수정해야 합니다. 예시 항목은 `title`, `content`, `category`, `favorite` 네 개의 키로 구성됩니다.

4.2 기능별 함수 구성

함수	담당 기능
<code>show_menu()</code>	메인 메뉴 출력
<code>input_required()</code>	빈 입력 방지
<code>select_category()</code>	카테고리 선택 및 직접 입력
<code>is_duplicate_title()</code>	같은 제목이 이미 등록되어 있는지 확인
<code>add_prompt()</code>	새 프롬프트 등록
<code>show_prompt_list()</code>	전체 목록 출력

함수	담당 기능
show_by_category()	카테고리별 조회
search_prompt()	제목과 내용 키워드 검색
show_detail()	전체 내용 상세 출력
toggle_favorite()	즐거찾기 추가-해제
show_favorites()	즐거찾기 목록 출력
save_prompts() / load_prompts()	JSON 저장 및 불러오기
export_markdown()	카테고리별 Markdown 내보내기

함수 구분	입력 또는 기준	결과
입력 함수	안내 문장·사용자 입력	입력한 글 또는 선택한 카테고리를 돌려줌
추가·변경 함수	제목·내용·번호	prompts 목록과 JSON 파일을 변경함
조회 함수	카테고리·검색어·번호	목록·검색·상세 결과를 화면에 출력함

예를 들어 input_required(안내 문장)는 사용자가 입력한 비어 있지 않은 글을 돌려줍니다.
save_prompts()는 반환값 대신 prompts.json 파일을 바꾸고, show_prompt_list()는 반환값 대신 목록을 화면에 출력합니다.

5. 필수 기능 구현 및 실행 결과

5.1 메뉴 선택과 프로그램 실행

프로그램을 실행하면 1 번부터 8 번까지의 기능 메뉴와 0 번 종료 메뉴가 표시됩니다. 사용자가 번호를 입력하면 해당 기능이 실행되며, 작업이 끝나면 다시 메인 메뉴로 돌아옵니다. 잘못된 번호를 입력하면 안내 문장이 표시되고 메뉴가 다시 나타납니다.

메뉴에 없는 번호는 안내 후 메뉴를 다시 표시하고 새 번호를 입력받습니다. 프롬프트 번호는 1 부터 전체 프롬프트 개수까지만 사용할 수 있습니다. 글자를 입력하면 숫자 입력 안내가 표시되며, 목록에 없는 번호는 잘못된 번호로 처리합니다. JSON 파일이 없을 때는 기본 프롬프트가 들어 있는 새 파일을 만들어 프로그램이 계속 실행되도록 하였습니다.

입력 범위 메인 메뉴는 0~8 만 허용합니다. 9 를 입력하면 '잘못된 번호입니다. 다시 입력해주세요.'라고 안내한 뒤 메뉴를 다시 보여줍니다.

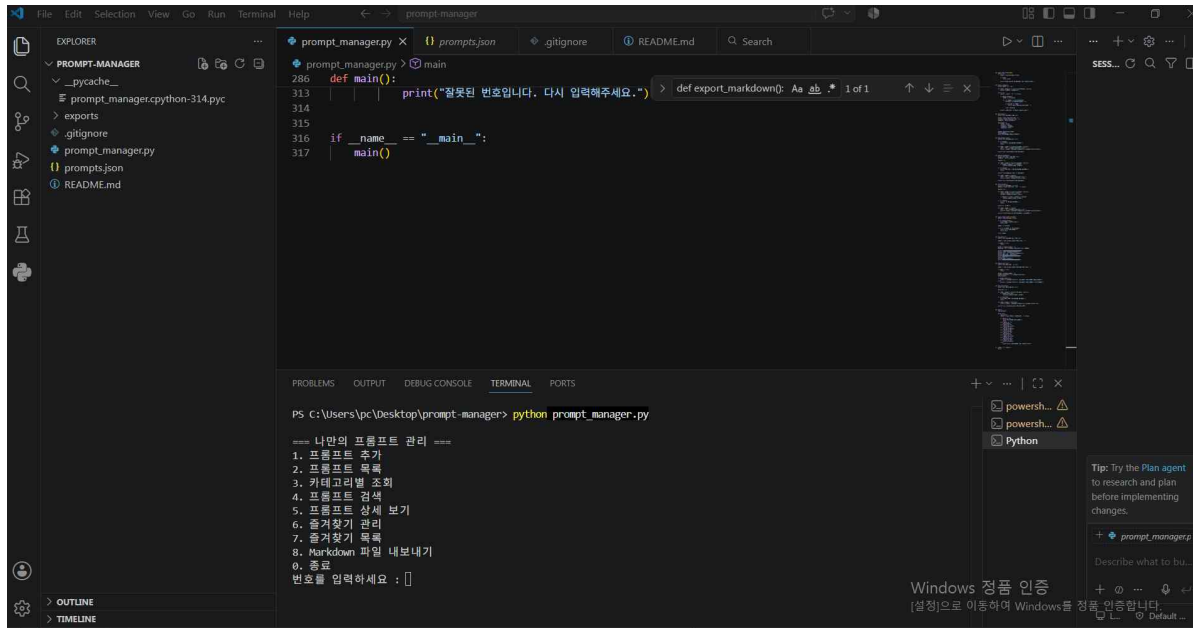


그림 3. 번호 입력 방식의 메인 메뉴 실행 화면

주석 프로그램을 실행하면 1 번부터 8 번까지의 기능과 0 번 종료 메뉴가 표시됩니다. 사용자는 원하는 번호를 입력하여 기능을 선택할 수 있습니다.

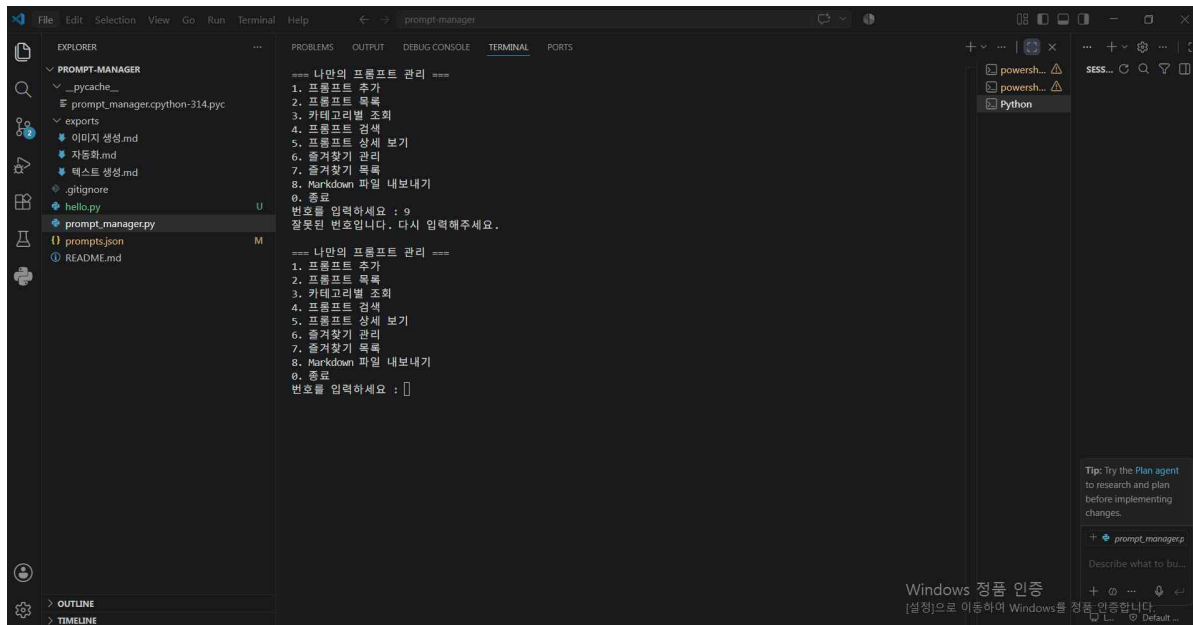


그림 4. 잘못된 메뉴 번호 입력 안내와 메뉴 재표시 결과

주석 메뉴에 없는 9 번을 입력한 화면입니다. '잘못된 번호입니다'라는 안내가 표시된 뒤 메인 메뉴가 다시 나타나므로, 잘못된 입력이 들어와도 프로그램이 계속 실행되는 것을 확인하였습니다.

5.2 프롬프트 추가 및 목록 보기

제목, 내용, 카테고리를 입력하면 새 프롬프트가 등록됩니다. 아무것도 입력하지 않으면 다시 입력하라는 안내가 표시됩니다. 새 프롬프트는 처음에는 즐겨찾기하지 않은 상태(False)로 저장됩니다. 목록 화면에는 번호, 카테고리, 제목과 즐겨찾기 별표가 함께 표시됩니다.

제목, 내용과 카테고리에는 별도의 최대 길이와 허용 문자 제한을 두지 않았습니다. 다만 빈칸은 저장할 수 없으며, 같은 제목이 이미 있으면 다른 제목을 다시 입력해야 합니다.

입력 예시는 제목 '블로그 제목 생성', 내용 '주어진 키워드로 제목을 5 개 작성해주세요', 카테고리 '텍스트 생성'입니다.

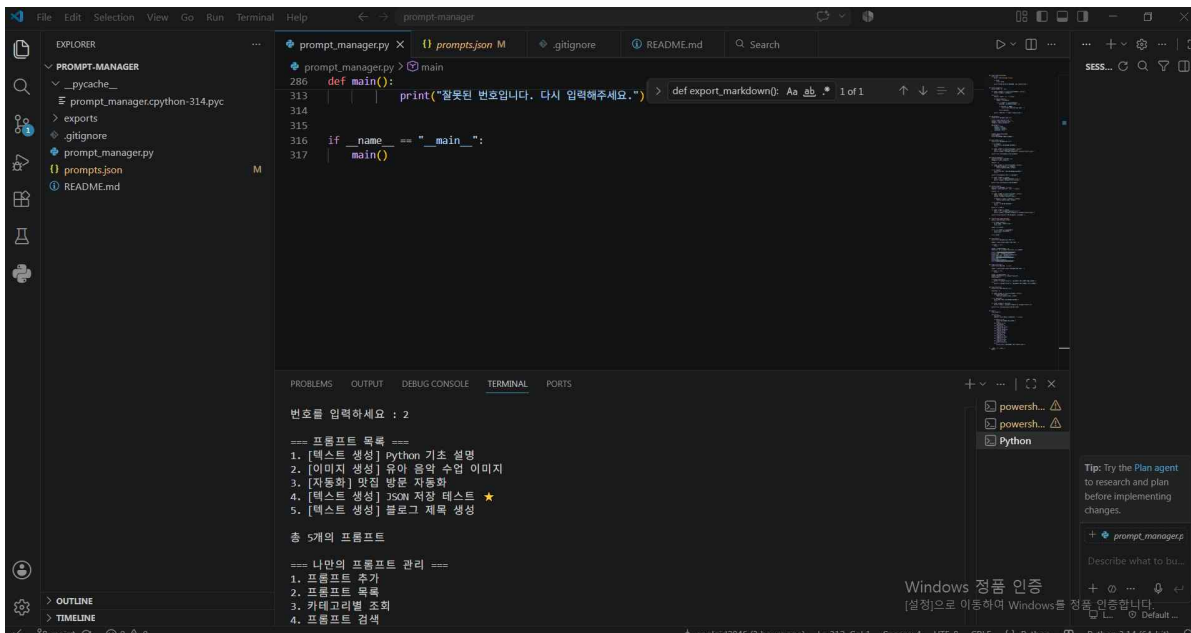


그림 5. 새 프롬프트가 전체 목록에 반영된 결과

주석 '블로그 제목 생성' 프롬프트를 추가한 뒤 전체 목록을 확인한 화면입니다. 새 프롬프트가 5 번으로 등록되었으며, 전체 개수도 5 개로 늘어났습니다.

중복 제목 방지 기능은 `prompt_manager.py` 의 `is_duplicate_title()`과 `add_prompt()`에서 처리합니다. 입력한 제목을 기존 프롬프트 제목과 비교하며, `lower()`를 사용하여 영어 대문자와 소문자를 같은 글자로 판단합니다. 같은 제목이 있으면 저장하지 않고 다른 제목을 다시 입력하도록 안내합니다.

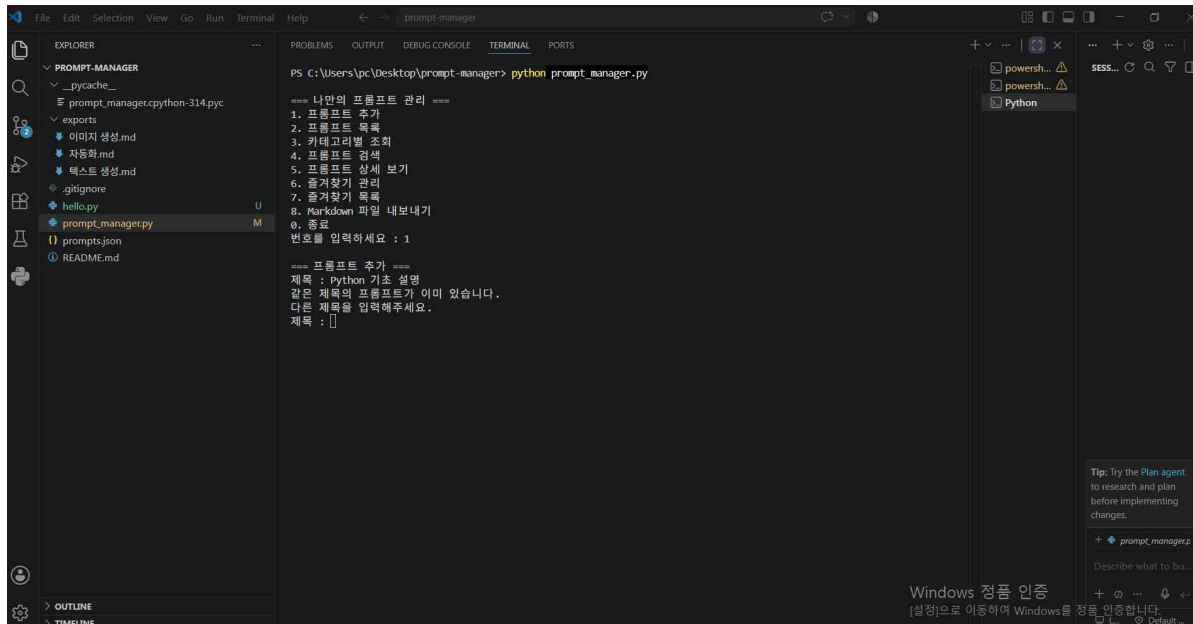


그림 6. 이미 등록된 제목의 중복 저장을 막는 실행 결과

주석 이미 등록된 'Python 기초 설명'을 다시 입력한 화면입니다. 같은 제목이 있다는 안내가 표시되고 다른 제목을 다시 입력하도록 처리되어 중복 제목이 저장되지 않습니다.

5.3 카테고리별 조회

카테고리 목록에서 번호를 선택하면 해당 카테고리의 프롬프트만 모아서 보여줍니다. 텍스트 생성 카테고리를 선택한 결과, 해당 프롬프트 3 개가 표시되었습니다.

선택한 카테고리 이름과 프롬프트의 category 값이 정확히 같을 때만 결과에 포함됩니다.

'기타'를 선택하면 사용자가 직접 입력한 카테고리 이름을 그대로 저장하고 조회합니다.

선택한 카테고리에 속한 프롬프트가 하나도 없으면 '해당 카테고리의 프롬프트가 없습니다.'라고 안내하고 메인 메뉴로 돌아갑니다.

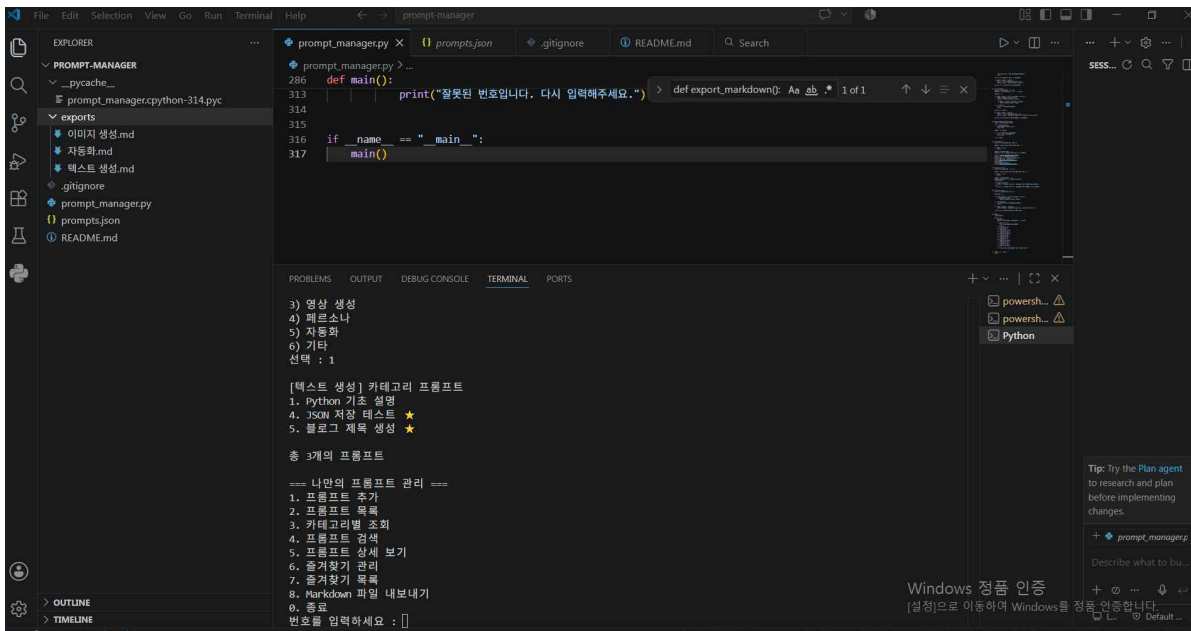


그림 7. 텍스트 생성 카테고리별 조회 결과

주석 텍스트 생성 카테고리를 선택한 화면입니다. 해당 카테고리에 들어 있는 프롬프트 3 개만 표시되어 카테고리별 조회 기능이 정상적으로 작동하는 것을 확인하였습니다.

5.4 프롬프트 검색

영어 검색어는 대문자와 소문자를 따로 구분하지 않도록 모두 소문자로 바꾼 뒤 찾습니다. 입력한 말이 프롬프트의 제목이나 내용에 들어 있으면 검색 결과에 표시됩니다. '블로그'를 검색한 결과, 프롬프트 2 개가 검색되었습니다.

검색 로직은 prompt_manager.py 의 search_prompt()에 있습니다. 검색어가 제목이나 내용의

일부에 들어 있어도 찾을 수 있는 부분 일치 방식입니다. lower()로 영어 대소문자를 같게 비교하지만, 불용어 제거와 정규식 검색은 사용하지 않았습니다. 한글과 영어는 검색할 수 있지만 띄어쓰기, 특수문자, 오타를 자동으로 고쳐주지는 않습니다.

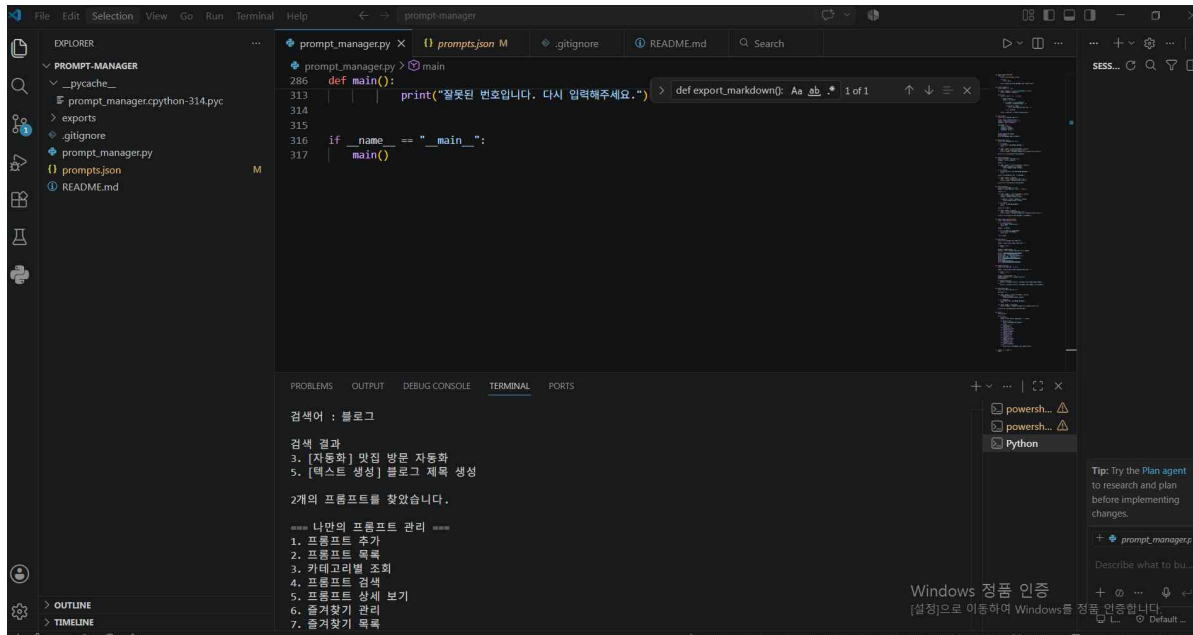


그림 8. 제목 또는 내용 기준 키워드 검색 결과

주석 검색어로 '블로그'를 입력한 화면입니다. 제목이나 내용에 '블로그'가 들어 있는 프롬프트 2 개가 검색되었습니다.

5.5 프롬프트 상세 보기

프롬프트 번호를 입력하면 제목, 카테고리, 즐겨찾기 여부와 내용 전체가 구분선과 함께 표시됩니다. 글자를 입력하거나 목록에 없는 번호를 입력하면 잘못된 입력이라는 안내가 표시된 뒤 메뉴로 돌아갑니다.

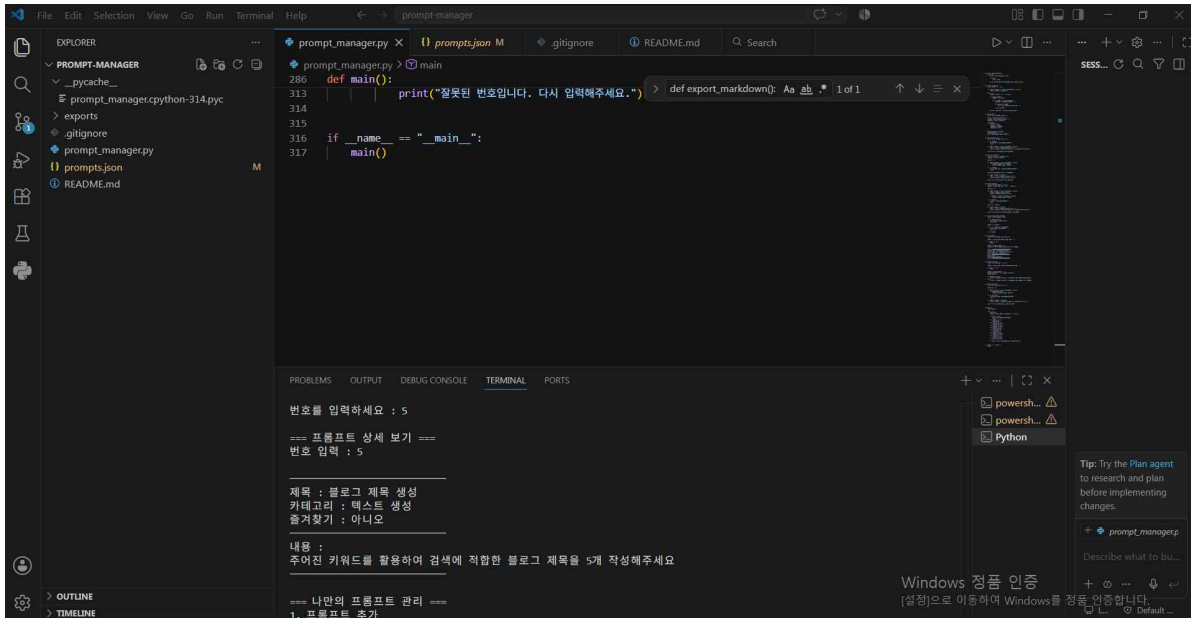


그림 9. 블로그 제목 생성 프롬프트의 상세 보기

주석 5 번 프롬프트의 상세 내용을 확인한 화면입니다. 제목, 카테고리, 즐겨찾기 여부와 전체 내용이 함께 표시되었습니다.

5.6 즐겨찾기 관리

즐거찾기를 선택하면 favorite 값이 False 에서 True 로 바뀝니다. False 는 즐겨찾기하지 않은 상태이고, True 는 즐겨찾기한 상태를 뜻합니다. 즐겨찾기 목록에서는 True 인 프롬프트만 별표와 함께 표시됩니다.

즐거찾기 상태가 바뀌면 추가 또는 해제되었다는 확인 메시지가 표시됩니다. 같은 프롬프트 번호를 다시 선택하면 이전 상태로 되돌릴 수 있습니다.

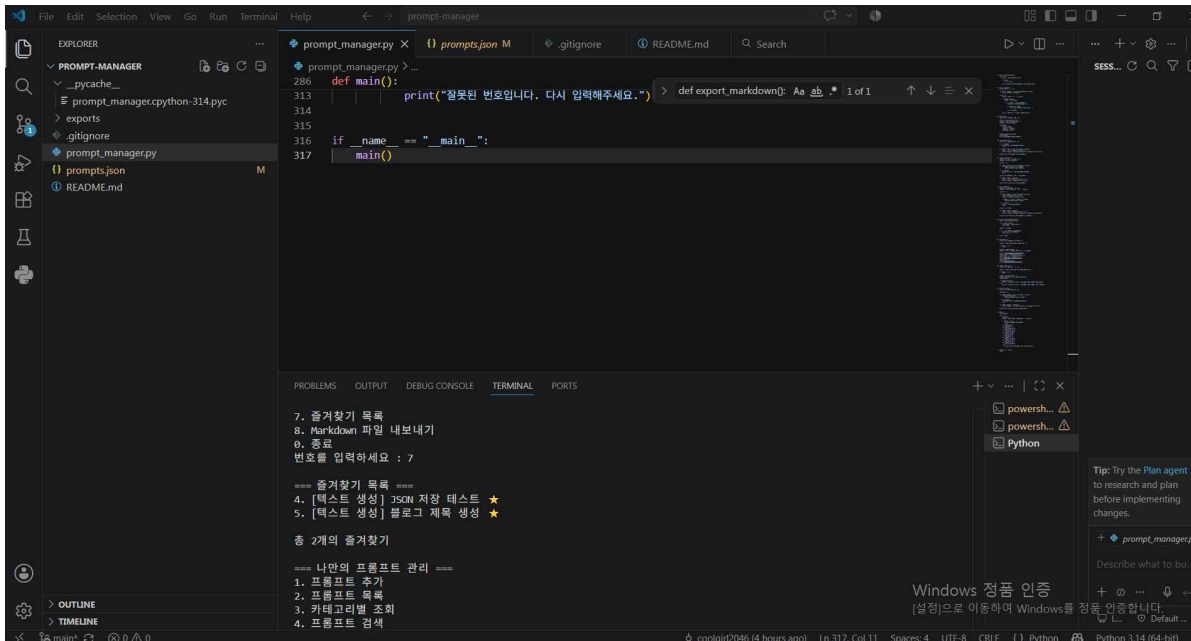


그림 10. 즐겨찾기 프롬프트만 모아서 출력한 결과

주석 즐겨찾기로 선택한 프롬프트 2 개가 별표와 함께 표시된 화면입니다. 즐겨찾기에 추가한 프롬프트만 모아서 확인할 수 있습니다.

6. 보너스 기능 구현

6.1 JSON 저장 및 불러오기

Python 에 기본으로 들어 있는 json 기능을 사용하여 프롬프트를 prompts.json 파일에 저장하였습니다. 프로그램을 시작하면 load_prompts() 함수가 저장된 내용을 읽습니다. 파일이 없으면 기본 프롬프트가 들어 있는 새 파일을 만듭니다. 프롬프트를 추가하거나 즐겨찾기를 변경하면 save_prompts() 함수가 바뀐 내용을 바로 저장합니다.

비교 항목	JSON	CSV
저장 구조	리스트와 딕셔너리를 그대로 저장	행과 열로 저장
긴 내용 저장	여러 줄의 프롬프트를 관리하기 쉬움	줄바꿈이 있으면 관리가 불편함
True-False	참·거짓 값으로 구분하여 저장	문자열처럼 보일 수 있음
프로젝트 적합성	제목·내용·카테고리·즐거찾기 저장에 적합	이 프로젝트에서는 상대적으로 불편함
선택 결과	사용	사용하지 않음

- 프로그램을 종료한 후 다시 실행해도 추가한 프롬프트가 유지됩니다.
- 프로그램을 종료한 후 다시 실행해도 즐겨찾기 상태가 유지됩니다.
- UTF-8 과 ensure_ascii=False 를 사용하여 한글이 깨지지 않게 저장하였습니다.
- indent 값을 사용하여 JSON 파일을 줄마다 보기 쉽게 정리하였습니다.

0 번을 눌러 종료할 때 따로 저장 명령을 실행하지는 않습니다. 프롬프트를 추가하거나 즐겨찾기를 바꾸는 시점에 이미 JSON 파일에 즉시 저장되기 때문에 종료 후에도 내용이 남습니다.

6.2 카테고리별 Markdown 내보내기

Python 의 Path 기능을 사용하여 exports 폴더를 만들었습니다. 프롬프트를 카테고리별로 나눈 뒤 카테고리 이름으로 Markdown 파일을 만들었습니다. 각 파일에는 프롬프트 제목, 즐겨찾기

별표와 내용 전체가 기록됩니다.

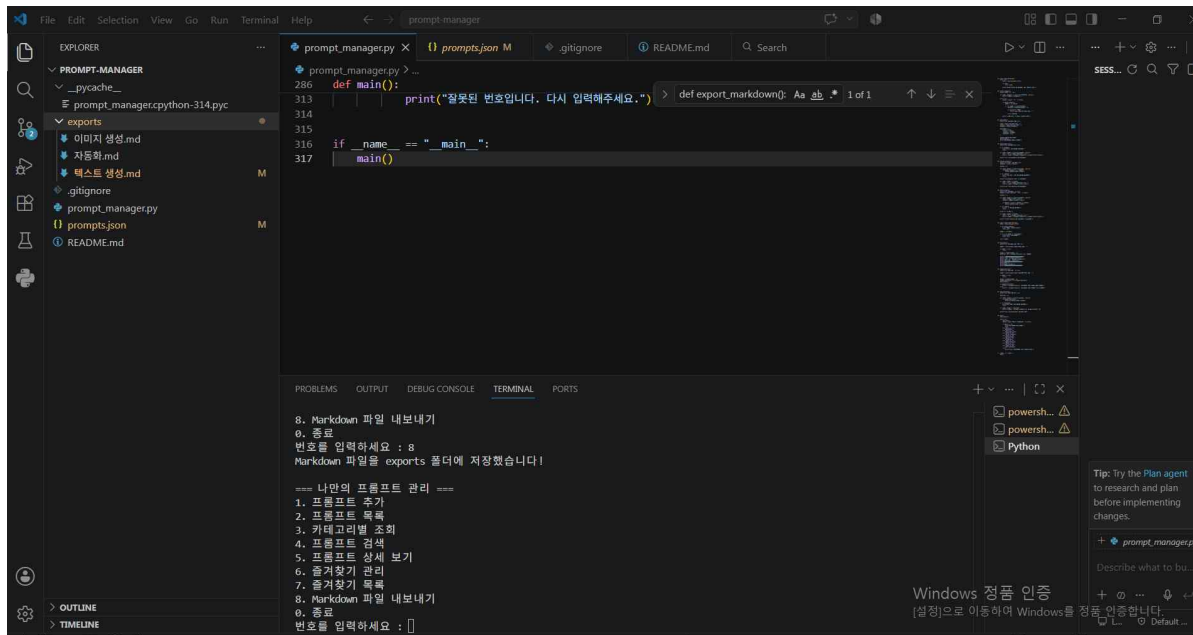


그림 11. exports 폴더와 카테고리별 Markdown 파일 생성 결과

주석 8 번 메뉴를 실행하여 exports 폴더를 만든 화면입니다. 폴더 안에 이미지 생성, 자동화, 텍스트 생성으로 나눈 Markdown 파일 3 개가 생성되었습니다.

7. Git 작업 기록 및 브랜치 병합

프로젝트 폴더에서 Git 기록을 시작한 뒤, 기능을 하나씩 만들 때마다 커밋으로 저장하였습니다. 프롬프트 목록 기능은 feature/prompt-list 브랜치에서 작업하였습니다. 보너스 기능은 feature/json-storage 브랜치에서 작업한 뒤 main 브랜치에 병합하였습니다. 병합된 기록은 Git 로그의 선 모양으로 확인할 수 있습니다.

커밋은 작업 내용을 한 번 저장한 기록입니다. 브랜치는 main 과 분리된 별도의 작업 공간이며, 병합은 브랜치에서 만든 기능을 main 에 합치는 과정입니다.

프롬프트 목록 브랜치는 `git checkout -b feature/prompt-list`, JSON 저장 브랜치는 `git checkout -b feature/json-storage` 명령으로 만들었습니다. 기능을 확인한 뒤 `git checkout main` 으로 main 에 돌아와 `git merge` 브랜치이름 순서로 합쳤습니다.

작업 항목	기준	프로젝트 적용
브랜치 이름	feature/기능이름	feature/prompt-list, feature/json-storage
병합 전 확인	기능 실행 후 정상 작동 확인	확인 후 main 브랜치에 병합
커밋 메시지	feat:, data:, chore:, merge:로 목적 표시	feat: prevent duplicate prompt titles 등
실제 병합 결과	충돌 여부 확인	충돌 없이 정상 병합

충돌 발생 시 순서	처리 방법
1. 원인 확인	충돌 파일과 충돌 표시 부분을 확인합니다.
2. 내용 수정	필요한 코드를 남기고 충돌 표시를 삭제합니다.
3. 기능 검증	프로그램을 다시 실행하여 정상 작동을 확인합니다.
4. 해결 기록	<code>git add</code> 와 <code>git commit</code> 으로 해결 내용을 저장합니다.

실제 예시로 GitHub 의 README 와 로컬 README 가 같은 부분에서 달라 `pull --rebase` 과정에서 충돌이 발생하였습니다. 원인을 확인한 뒤 `rebase` 를 중단하고, 잘못 수정한 README 커밋만 취소하였습니다. 그런 다음 GitHub 의 최신 내용을 fast-forward 방식으로 가져오고 `git status` 로

main 과 origin/main 이 같은 상태임을 확인하였습니다.

주요 커밋 예시는 'feat: prevent duplicate prompt titles'(중복 제목 방지), 'data: add blog prompt example'(예시 데이터 추가), 'chore: remove Python cache from repository'(캐시 정리), 'merge: add bonus persistence features'(보너스 브랜치 병합)입니다.

명령어	수행 내용	상태
git init	현재 폴더에서 Git 기록 시작	완료
git add	저장할 변경 파일 선택	완료
git commit	기능 단위 변경 이력 기록	완료
git checkout	feature 브랜치 및 main 이동	완료
git merge	기능 브랜치를 main 에 병합	완료
git push	main 브랜치를 GitHub 에 업로드	완료
git pull	GitHub 의 최신 내용이 있는지 확인하고 가져오기	완료
git clone	공개 Hello-World 저장소를 내려받아 폴더와 기록 확인	완료

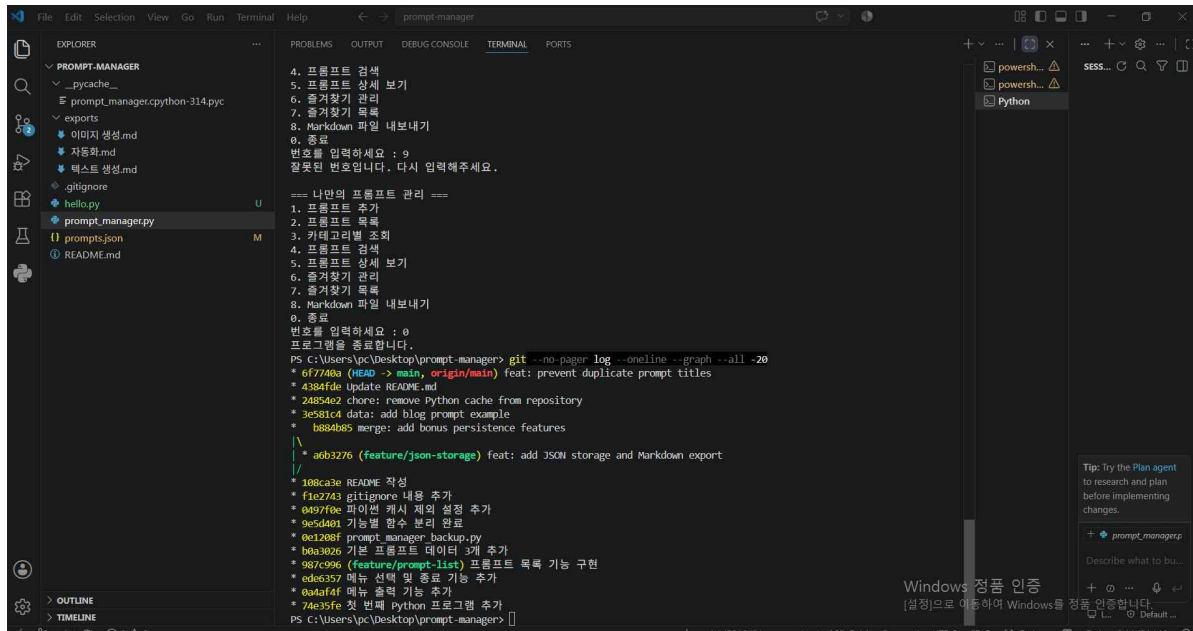


그림 12. 촬영 당시 16 개 커밋과 기능 브랜치 병합이 표시된 Git 그래프

주석 Git 기록을 확인한 화면입니다. 스크린샷 촬영 당시 중복 제목 방지 기능을 포함한 커밋 16 개와 기능 브랜치:main 브랜치의 병합선을 확인하였습니다. 이후 README 보완 커밋이 추가되어 실제 커밋 수는 더 많습니다.

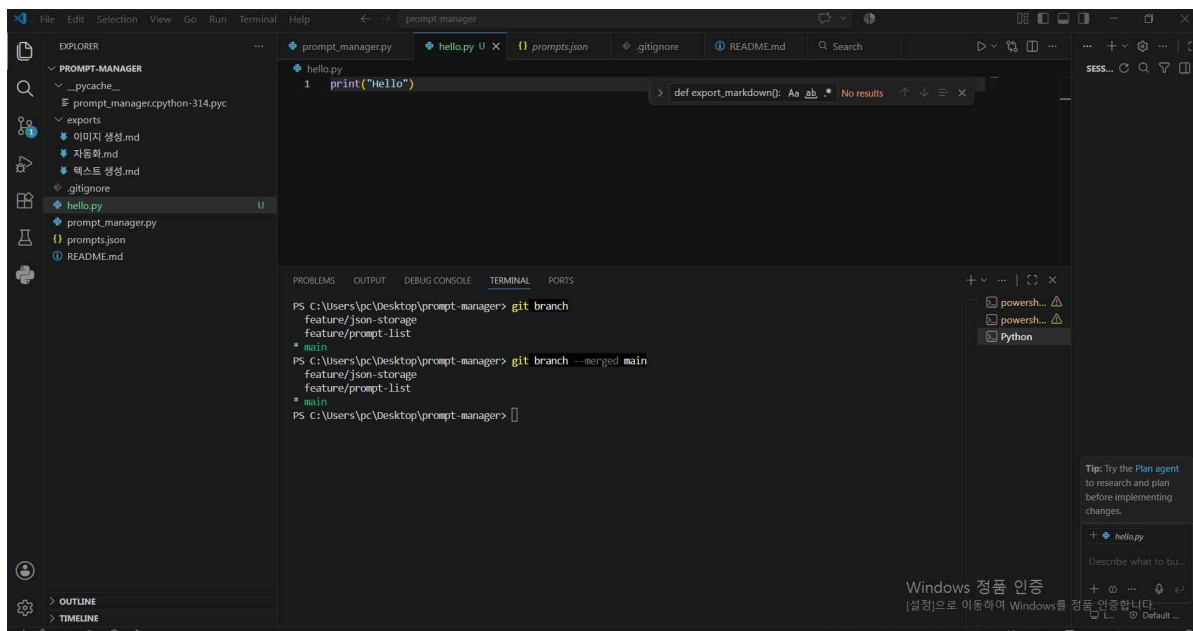


그림 13. 기능 브랜치 생성과 main 병합 확인

주석 기능 브랜치 2 개와 main 브랜치를 확인한 화면입니다. 두 기능 브랜치가 main 에 모두 병합되었다는 결과도 함께 확인하였습니다.

공개 샘플 저장소는 git clone 명령으로 sample-repository 폴더에 내려받았습니다. 다운로드 후 dir 에서 README 파일을, git log --oneline 에서 7fd1a60, 7629413, 553c207 커밋을 확인하였습니다.

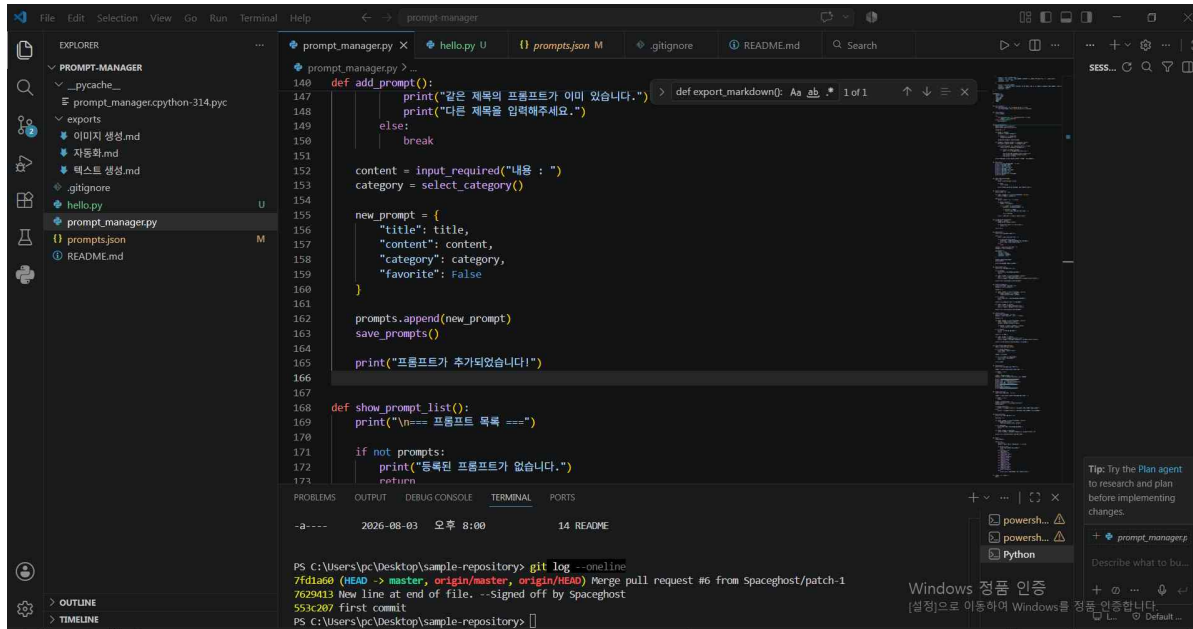


그림 14. 공개 샘플 저장소 clone 후 폴더와 Git 로그 확인

주석 GitHub의 공개 샘플 저장소를 clone 한 결과입니다. sample-repository 폴더의 README 파일과 git log 명령으로 확인한 커밋 3 개가 함께 표시되어 있습니다.

8. GitHub 저장소 업로드 결과

최종 코드는 누구나 확인할 수 있는 GitHub 저장소의 main 브랜치에 업로드하였습니다. 주요 파일은 실행 코드인 `prompt_manager.py`, 프로그램 설명서인 `README.md`, 제외 규칙을 적은 `.gitignore`, 저장 데이터인 `prompts.json`, 내보내기 결과인 `exports` 폴더입니다. Python 이 자동으로 만드는 `__pycache__` 폴더는 GitHub 에 올라가지 않도록 제외하였습니다.

저장소 URL <https://github.com/coolgirl2046/prompt-manager>

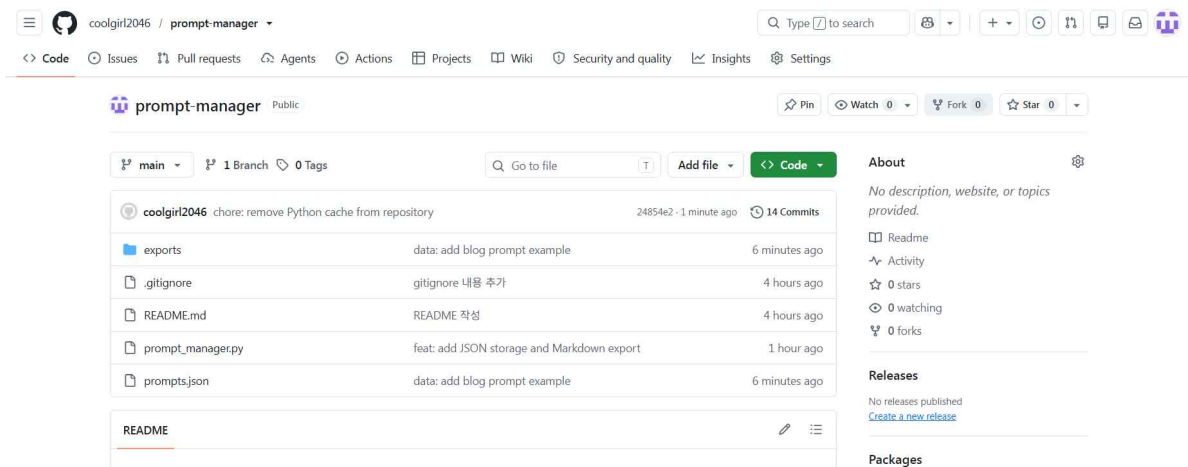


그림 15. GitHub 저장소의 주요 파일 구성

주석 GitHub 의 `prompt-manager` 저장소에 결과물을 올린 화면입니다. `exports` 폴더, `README.md`, `prompt_manager.py` 와 `prompts.json` 등 주요 파일 구성을 확인할 수 있습니다. 최신 커밋 수는 별도의 Git 로그 화면에서 확인합니다.

9. 최종 제출 증빙 요약

AI가 그림 속 글자를 읽지 못해도 실행 결과를 확인할 수 있도록 아래 표에 화면 내용을 글로 정리하였습니다. 실행한 명령이나 입력값, 화면에 표시된 결과와 확인한 과제 항목을 순서대로 확인할 수 있습니다.

9.1 그림 1~8 증빙

그림	실행·입력	화면에 표시된 핵심 결과	충족 항목
그림 1	python --version git --version git config --global user.name git config --global user.email	Python 3.14.6 git version 2.55.0.windows.3 user.name: coolgirl2046 user.email: coolgirl2046@naver.com	Python 3.10 이상 Git 사용자 설정
그림 2	hello.py 에 print("Hello") 작성 python hello.py 실행	터미널에 Hello 출력	Python 파일 생성·실행
그림 3	python prompt_manager.py	1~8 번 기능 메뉴와 0 번 종료 메뉴 출력	콘솔 메뉴 실행
그림 4	메뉴 번호: 9	잘못된 번호 안내 후 메인 메뉴가 다시 표시됨	잘못된 입력 처리 반복 실행
그림 5	1 번 추가 후 2 번 목록	'블로그 제목 생성'이 5 번으로 추가되고 총 5 개 출력	프롬프트 추가·목록
그림 6	1 번 추가 제목: Python 기초 설명	같은 제목 안내 후 다른 제목을 다시 입력받음	중복 제목 방지
그림 7	3 번 카테고리 조회 1 번 텍스트 생성 선택	Python 기초 설명, JSON 저장 테스트, 블로그 제목 생성 등 총 3 개 출력	카테고리별 조회
그림 8	4 번 검색 검색어: 블로그	맛집 방문 자동화와 블로그 제목 생성 등 2 개 검색	제목·내용 부분 검색

9.2 그림 9~15 증빙

그림	실행·입력	화면에 표시된 핵심 결과	충족 항목
그림 9	5 번 상세 보기 프롬프트 번호: 5	제목, 카테고리, 즐겨찾기 여부와 프롬프트 내용 전체 출력	상세 보기·번호 검증
그림 10	6 번 즐겨찾기 관리 후 7 번 목록	JSON 저장 테스트와 블로그 제목 생성 등 즐거찾기 2 개 출력	즐거찾기 추가·해제·목록
그림 11	8 번 Markdown 파일 내보내기	'Markdown 파일을 exports 폴더에 저장했습니다!' 출력 및 카테고리 파일 3 개 생성	보너스 Markdown 내보내기
그림 12	git --no-pager log --oneline --graph --all -20	촬영 당시 16 개 커밋과 브랜치 병합선 표시, 이후 README 보완 커밋 추가	10 개 이상 커밋 브랜치·merge
그림 13	git branch git branch --merged main	feature/json-storage, feature/prompt-list, main 이 두 결과에 모두 표시	추가 브랜치 생성 main 병합 완료
그림 14	공개 저장소 clone 후 dir git log --oneline	README 파일과 공개 저장소 커밋 3 개 확인	git clone 실행 증거
그림 15	GitHub 저장소 접속	main 브랜치에 exports, .gitignore, README.md, prompt_manager.py 와 prompts.json 표시	GitHub 업로드 README·저장소 URL